Programming Language and Paradigms

Feb. 2025

Course Context

- What are programming languages?
- Programming language concepts
- Programming languages design Principles
- Programming language implementation techniques
- Why paradigms matter?
- How modern languages address challenges?
  - Scalability, Concurrency, Safety, Security, and distributed computing

Chapter 1

Principles of Programming Language Design

This chapter sets the foundation for the entire course. Before we write code, we must understand what programming languages really are, why there are so many of them, and how different ways of thinking about programming evolved over time.

Programming Languages as Computational Notations

- A programming language is a *formal notation* for expressing computation
  - Formal:
    - Precise, Unambiguous, Governed by strict rules, Machine-checkable
    - Natural languages are mostly ambiguous, context-dependent, and open to interpretation
    - In a formal language, every valid program has a well-defined meaning, and the invalid ones are rejected.
  - Notation
    - A system of symbols used to represent ideas (mathematics, music, logic)
    - Programming languages are notations for algorithms.
  - Computation
    - Transforming inputs into outputs, according to well-defined rules, and in finite steps
  - Programming ≠ typing code (thinking, modeling, reasoning)
  - Different languages, same computation (different notations, same computation)

… cont'd

- Languages bridge human thinking and machine execution
  - Programming languages sit in the middle of Human thinking, and Machine execution
  - Human thinking (goals, concepts, abstractions, approximate logic)
  - Human thinking (high-level, conceptual, context-aware, imprecise)
  - Machine execution (binary values, exact instructions, deterministic steps, intent not aware)
- Bridging
  - Abstraction (Human → Language)
    - Variables instead of memory addresses
    - Loops instead of jumps
    - Functions instead of instruction sequences
    - Objects instead of data structures
  - Formalization
  - Translation

A programming language is not just a tool. It is a formal notation. Just like mathematics, it has rules, structure, and meaning. Its job is to translate human intentions into machine-executable actions.

Why Programming Languages Exist?

- Hardware abstraction
  - Languages are abstractions over hardware
  - Languages hide (CPU architecture, Memory layout, Instruction sets)
- Productivity
  - Why don't we program everything in assembly if it's faster?
- Reliability and portability
  - Reliable programs behave correctly; portable programs run everywhere. Good languages aim for both.
  - Portability (Hardware abstraction, Standard language specifications, virtual machines and runtimes)

Imagine writing every program in binary. Languages exist to abstract hardware complexity, increase productivity, reduce errors, and make programs portable across machines.

Levels of Programming Languages

- Machine language
- Assembly language
- High-level languages
- Domain-Specific Languages (DSLs)

- Languages exist at different abstraction levels
  - Abstraction level → how far a language is from the hardware (low vs high)
    - Machine Code → Assembly → C → Java / C# → Python

Languages exist at different abstraction levels. As we move up, we gain readability and lose direct hardware control. High-level languages trade raw speed for safety and productivity.

As we go up this ladder:

- Use more expressive constructs
- Gain readability (more expressive constructs, structured control flow, names instead of addresses, built-in data structures)
- Lose direct hardware control (hide hardware details)
  - Register manipulation, Memory layout, Pointer arithmetic, CPU-specific instructions
  - Low-level languages (decide how memory is used, optimize instruction-by-instruction)
  - High-level languages (hide registers, manage memory automatically, abstract CPU details)

As we go up this ladder:

- Trade raw speed
  - High-level languages often run on virtual machines, use garbage collection, perform run time checks, add abstraction overhead.
  - All of the above cost CPU cycles
  - C can be faster than Python, and assembly can be faster than C
- For safety and productivity
- Safety:
  - High-level languages increase safety by preventing invalid memory access, enforcing type rules, catching errors automatically
- Productivity
  - Fewer lines of code
  - Faster development
  - Easier debugging
  - Easier maintenance

Domain-Specific Languages

- SQL
- HTML
- MATLAB

DSLs are designed for specific problem domains. SQL doesn't describe how to get data, but what data you want—this is an early glimpse of declarative thinking.

Principles of Language Design

- Readability
  - How easily a human can understand a program by reading it.
  - Can another programmer understand what this code does quickly and correctly?
  - What improves readability?
    - Simple and consistent syntax
    - Clear keywords
    - Structured control flow
    - Meaningful identifiers
    - Fewer special cases
  - Why readability matters?
    - Reduces bugs
    - Improves maintainability
    - Makes team work possible
    - Lowers learning curve

Principles of Language Design

- Writability
  - How easily a programmer can express ideas in the language.
  - How much effort does it take to write correct and clear programs?
  - What improves writability?
    - High-level abstractions
    - Built-in data structures
    - Expressive syntax
    - Good libraries

Principles of Language Design

- Reliability
  - The degree to which a program behaves correctly under all expected conditions.
  - What improves reliability?
    - Strong type systems
    - Error checking
    - Exception handling
    - Memory safety
    - Restricted dangerous operations
  - Why reliability is critical?
    - Banking systems
    - Medical systems
    - Aviation systems
    - Security-critical software

Principles of Language Design

- Efficiency
  - Measures how well a language uses hardware resources.
    - Execution time (speed)
    - Memory usage
    - Power consumption
  - What improves efficiency?
    - Low-level control
    - Minimal runtime overhead
    - Direct hardware access
    - Manual memory management
  - Trade-off
    - Improving efficiency often reduces readability, writability, reliability

Good language design balances these four criteria. Improving one often hurts another. Language design is about trade-offs.

Programming Paradigms

- A way of thinking about computation
- Different paradigms emphasize different mental models, not just syntax.
  - Imperative
    - Procedural programming
    - State changes every iteration, and order of execution matters
  - Object-Oriented
    - A collection of interacting objects
    - Has state (data) and behavior (methods)
  - Functional
    - A program is the evaluation of mathematical functions.
  - Logic
    - A program is a set of facts and rules

A paradigm is not syntax. It's a mental model. Each paradigm shapes how we think about problems and solutions.

- Concurrent
  - Multiple computations happen at the same time.
  - Threads
  - Processes
  - Synchronization
  - Deadlock
  - Race conditions

  - Better performance on multi-core systems

  - Essential for servers and distributed systems

The same problem can be solved in many ways. Paradigms influence clarity, correctness, and maintainability, not just code style.

Application Domains

- Systems Programming
  - Programming that interacts directly with hardware or the operating system (C, C++, Rust)
- Scientific computing
  - Programming that interacts directly with hardware or the operating system (MATLAB, NumPy, SciPy)
- Enterprise
  - Large-scale business software systems (Java, C#, Python, Kotlin)
- AI & data science
  - Programming for learning from data and intelligent behavior (Python (dominant), R)

Languages evolve to serve domains. There is no universally best language, only a best match for a problem.

Evolution of Software Architecture

- Monolithic
  - A single, tightly coupled program where:
    - All components live in one codebase
    - All parts are deployed together
    - One failure can affect the whole system
- Modular
  - The system is divided into modules with clear responsibilities.
    - Each module handles one concern
    - Modules interact via well-defined interfaces
- Layered
  - A specialized form of modularity where modules are arranged in layers.
    - Has a specific responsibility
    - Uses only the layer below it
- Microservices
  - The system is split into small, independent services that:
    - Run as separate processes
    - Communicate over the network
    - Can be developed and deployed independently
    -

As software systems grew, architectures evolved, and languages evolved with them to manage complexity.

… cont'd

Criteria	PHP	Java	C#	Python	JavaScript /NodeJS	Ruby
Development cost	Expensive	Cheaper	Cheaper	Cheapest	Cheapest	-
Deployment cost	Cheapest	Medium	Expensive	Medium	Medium	-
Ease of Deployment	Easiest	Easier	Less easy	Easy	Easier	-
Security	Less secure	Secure	Secure	Secure	Secure	-
Performance	Best	Better	Good	Good	Better	-
Learning curve	Easier	Easy	Medium	Easiest	Medium	-
Popularity	Popular	Popular	Popular	Popular	Popular	-
Framework support	Yes - many	Yes – some	Yes – some	Yes – many	Yes-many	-
Area of Usage	Anywhere	Europe and US	US	Europe	Asia, ?	-
Free & Open?	Yes	Yes	No	Yes	Yes	-

… cont'd

PL / Platform	Web frameworks	JavaScript UI Frameworks	UI Frameworks	DBMS	Web API
PHP	Laravel	Vue	Vuetify ~~/Bootstrap~~	MariaDB/MongoDB	REST
Java	Spring/JSF	Angular	Material	MySQL	REST
NodeJS	Express	Vue	Vuetify	MySQL /MongoDB	REST
Python	Django	React	Bootstrap	MySQL	REST
C#	dotNet	Angular	Kendo	MSSQL	REST
Ruby	Ruby on rails	Angular	Bootstrap	PostgreSQL	REST

End!